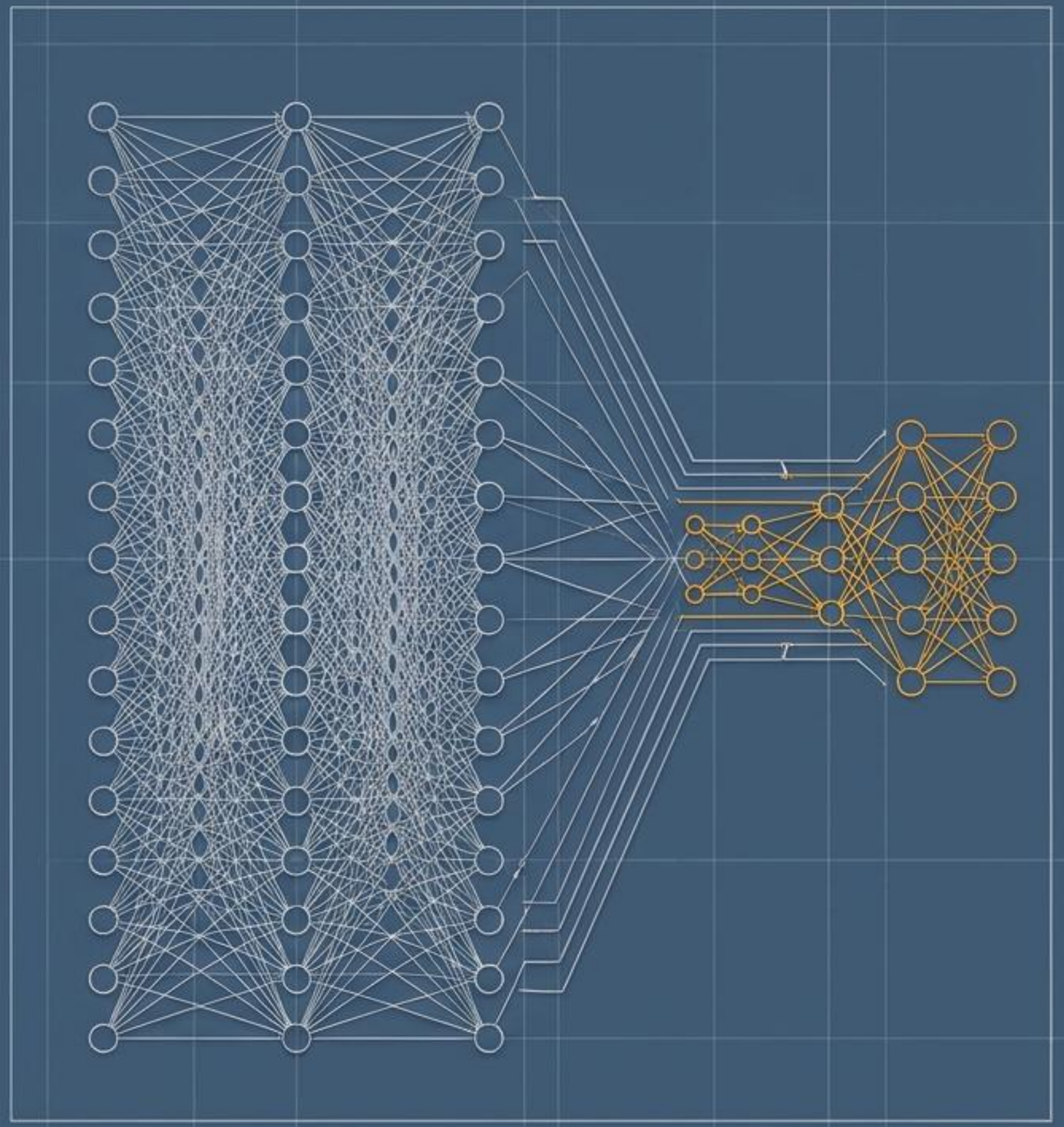
TRANSFER LEARNING

Fundamentos y Feature Extraction

Curso IS056B — Ingeniería del Conocimiento

UNCP 2026-II | **Unidad II:** Transferencia de Aprendizaje

Docente: Msc. Jaime Antonio Huaytalla Pariona



El muro computacional de las CNNs desde cero

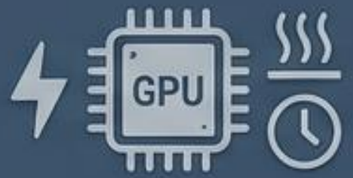
Caso de Estudio: Clasificación de papa nativa (Valle del Mantaro) con PlantVillage (38 clases)

Fuerza Bruta — Sesión 08



Requiere miles de imágenes por clase para generalizar.

DATA_REQ > 10k



Alta demanda computacional: Horas/Días de GPU y alto costo energético.

GPU_TIME: HIGH

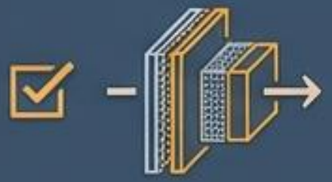


Alto riesgo de overfitting ante datasets pequeños (memorización).

DATA_REQ > 10k

Resultado: 85-90% de accuracy en 2 horas.

Conocimiento Previo — Sesión 09



Aprovecha filtros visuales pre-aprendidos en millones de imágenes.

PRETRAINED_MODELS: LOADED



Entrenamiento ultra-rápido de solo la capa final.

FREEZE_LAYERS: TRUE



Resistencia al overfitting en datasets reducidos.

TRAIN_TOP: ONLY

¿Podemos lograr >95% de accuracy en apenas 15 minutos? Sí.

Aprovechando los hombros de gigantes: ¿Qué es Transfer Learning?

Analogy



Radiólogo
(20 años exp. en tórax)

Transfiere: Anatomía base y lectura de contrastes



Aprende la particularidad ósea
(No necesita volver a estudiar medicina)

Technical Diagram

Source Domain (Ds): ImageNet

DATA_SRC > 1M

1.2M fotos, 1000 clases.

Aprende filtros universales: bordes, texturas, formas orgánicas.

FILTERS: UNIVERSAL

Transferencia de Conocimiento

Target Domain (Dt): Hojas Enfermas

DATA_TGT: PLANTVILLAGE

Dataset PlantVillage (38 clases).

Reutiliza los filtros universales para este problema específico.

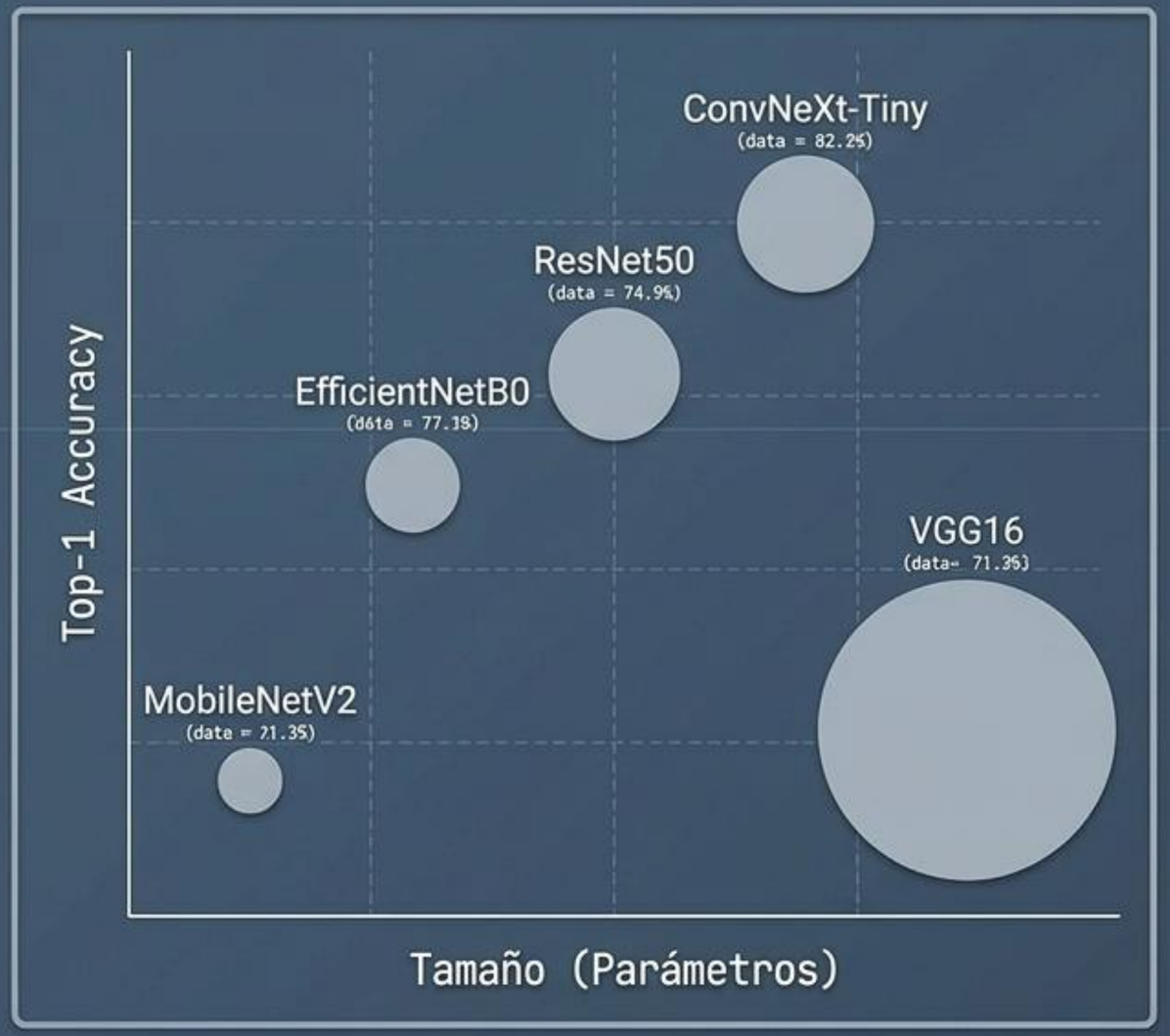
LAYERS: REUSED + TRAINABLE

Las 4 estrategias de Transferencia de Aprendizaje

Complejidad y Requerimiento de Datos

Feature Extraction (Enfoque de hoy)	Pesos congelados + nuevo clasificador. `FREEZE_LAYERS=TRUE, NEW_CLASSIFIER=TRUE`	Uso: Dataset pequeño (<5K imgs), dominio similar. `DATASET_SIZE<5K, DOMAIN_SIMILARITY=HIGH`
Fine-Tuning Parcial	Descongelar últimas N capas profundas. `UNFREEZE_LAYERS=N, LR=LOW`	Uso: Dataset mediano (5K-50K imgs). `DATASET_SIZE=5K-50K, DOMAIN_SIMILARITY=MEDIUM`
Fine-Tuning Completo	Descongelar toda la red (LR muy bajo). `UNFREEZE_LAYERS=ALL, LR=VERY_LOW`	Uso: Dataset grande (>50K imgs), dominio distinto. `DATASET_SIZE>50K, DOMAIN_SIMILARITY=LOW`
Domain Adaptation	Alinear distribuciones estadísticas. `ALIGN_DISTRIBUTIONS=TRUE, DOMAIN_SHIFT=SIGNIFICANT`	Uso: Dominios radicalmente distintos (ej. fotos a dibujos). `DOMAIN_DIFFERENCE=EXTREME, EXAMPLE=PHOTO_TO_SKETCH`

Nuestro arsenal pre-entrenado: tf.keras.applications



Modelo	Params	Tamaño	Top-1 Acc	Característica
VGG16	138M	528 MB	71.3%	Pedagógico, simple
ResNet50	25.6M	98 MB	74.9%	Skip connections
MobileNetV2	3.4M	14 MB	71.3%	Edge AI / Móviles
Efficient-NetB0	5.3M	29 MB	77.1%	Balance ideal
ConvNeXt-Tiny	28.6M	109 MB	82.1%	Estado del arte

Cirugía de Redes Neuronales: Backbone vs. Head

CLASIFICADOR ORIGINAL (Head)

`GlobalAvgPool → Dense(1000)`

Específico para ImageNet. Se elimina siempre.

BASE CONVOLUCIONAL
(Backbone)

`Conv → BN → ReLU → Pool`

Features aprendidos de ImageNet.
En Feature Extraction, esto se CONGELA.

Separando la cabeza del cuerpo

```
base = tf.keras.applications.ResNet50(
    weights='imagenet', # Extrae conocimiento
    include_top=False,  # Corta el clasificador
    input_shape=(224, 224, 3)
)
```

Feature Extraction (1/3): Cargar y Congelar el Backbone



Usar la red pre-entrenada como un **extractor de features estático**. Cero modificaciones a la experiencia original.

```
base_model = tf.keras.applications.EfficientNetB0(
    weights='imagenet',
    include_top=False
)

# CONGELAR LOS PESOS (El paso crítico)
base_model.trainable = False

# Comprobación de seguridad:
# Parámetros totales: ~4.0M
# Parámetros entrenables: 0
```

Feature Extraction (2/3): Ensamblar el Nuevo Clasificador




```
NUM_CLASSES = 38 # Clases de PlantVillage

model = models.Sequential([
    base_model,                                    # Extractor estático
    layers.GlobalAveragePooling2D(),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(NUM_CLASSES, activation='softmax') # Nuevo Head
])
```

Feature Extraction (3/3): El Peligro del Preprocesamiento



Cada modelo exige reglas numéricas distintas. Normalizar a ciegas destruye la inferencia.

Lo que NO se debe hacer

```
# ❌ INCORRECTO para la mayoría
# Asumir normalización estándar
x = x / 255.0
```

MobileNetV2 espera [-1, 1], ResNet50 usa BGR centrado, EfficientNet espera [0, 255]. Dividir por 255 arruina la entrada.

La Regla de Oro

```
# ✅ CORRECTO: Usar la función nativa
from tf.keras.applications.efficientnet
import preprocess_input

def prepare_dataset(ds):
  def preprocess(images, labels):
    return preprocess_input(images),
    labels
  return ds.map(preprocess)
```

El Retorno de Inversión (ROI) del Transfer Learning

Métrica	CNN Desde Cero (S08)	Feature Extraction (MobileNetV2)
Validation Accuracy	~85 - 90%	~92 - 95% (Mejora dramática)
Épocas hasta convergencia	30 - 50 épocas	5 - 10 épocas
Tiempo por época (GPU T4)	~3 - 5 min	~1 - 2 min
Parámetros a entrenar	~2.8 Millones	~100 Mil (Solo el 3%)



Precisión



Tiempo



Cómputo

Abriendo la Caja Negra: ¿Qué estamos transfiriendo?

Jerarquía de Features



Capas tempranas
Bordes y colores universales.

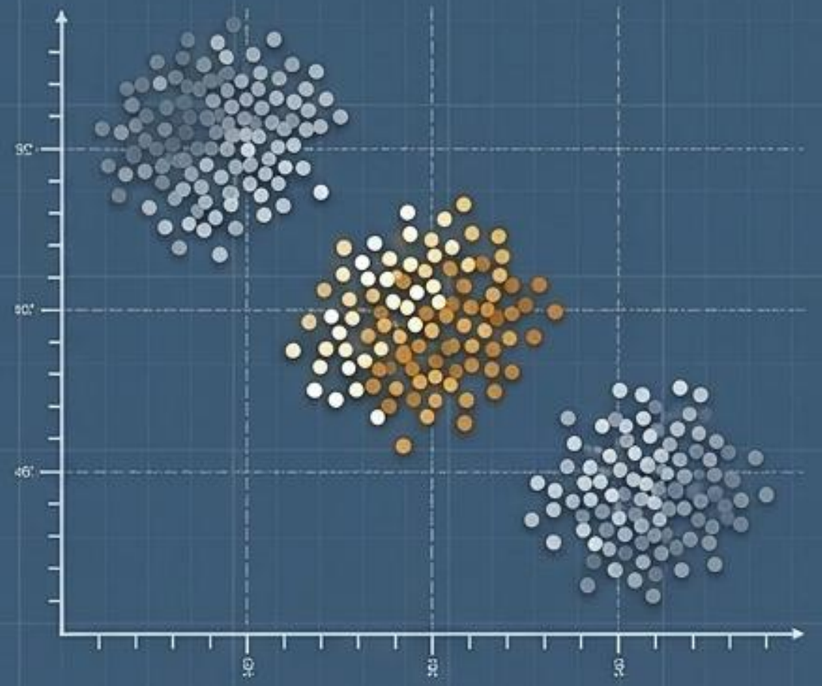


Capas intermedias
Texturas y patrones.



Capas profundas
Formas complejas y semántica.

Separación Lógica en el Backbone



El t-SNE demuestra que el backbone pre-entrenado de ImageNet ya agrupa lógicamente las hojas enfermas ANTES de entrenar nuestra capa clasificadora.

Matriz de Decisión Arquitectónica: ¿Qué modelo base elegir?



Servidor / Máxima Precisión

Modelos: EfficientNetV2-L, ConvNeXt-Base.

Uso: GPUs de alto rendimiento, donde cada decimal de accuracy es crítico.



Balance (Ej. Google Colab T4)

Modelos: EfficientNetB0-B3, ResNet50.

Uso: Excelente precisión sin saturar la memoria VRAM.



Edge AI / Despliegue Móvil

Modelos: MobileNetV2, MobileNetV3-Small.

Uso: Tamaño < 15MB. Baja latencia para análisis en campo.



Data Limitada (<1K imágenes)

Modelos: MobileNetV2.

Uso: Reducir parámetros totales para minimizar riesgo de overfitting.

El Reverso de la Moneda: Negative Transfer

Ocurre cuando forzar conocimiento previo empeora el modelo.

Causas Principales



Brecha de Dominio: ImageNet (perros/gatos) no transfiere bien a Radiografías médicas o Imágenes Satelitales.

Brecha de Resolución: Entrenado en 224x224 pero aplicado a Microscopía de 2048x2048.

Síntomas y Soluciones



Síntoma: Accuracy estancada en probabilidad aleatoria (ej. 1/N clases) desde la primera época. $f = \frac{1}{N}$



Solución A: Buscar modelos base especializados (ej. `CheXNet` para dominio médico).



Solución B: Fine-Tuning agresivo (descongelar el **backbone** entero).

Los 5 Pilares de la Sesión

1. Fundamento

Reutilizar features de un dominio rico acelera el aprendizaje radicalmente.





2. Anatomía

Separar rígidamente el Backbone (extractor) del Head (clasificador).





5. Criterio

El mejor modelo es el que se ajusta a tu hardware, RAM y volumen de datos.



2. Anatomía

Separar rígidamente el Backbone (extractor) del Head (clasificador).

3. Ejecución

Para Feature Extraction: `include_top=False` y `trainable=False`.



4. Seguridad

Jamás normalizar a ciegas. Usar el preprocess_input específico.

5. Criterio

El mejor modelo es el que se ajusta a tu hardware, RAM y volumen de datos.

Próximamente (S09-B): Desbloqueando el Fine-Tuning

El Límite de Hoy:

Hemos construido sobre cimientos fijos. ¿Qué pasa si esos cimientos necesitan sutiles ajustes para nuestro dominio?



Próxima Sesión:

Aprenderemos a descongelar bloques profundos del backbone usando Learning Rates diferenciados.

Tarea de Investigación: Apliquen la estrategia de Feature Extraction a sus proyectos de la Sesión 08. Documenten el delta de precisión y ahorro de tiempo en GPU.